2026-05-08-p3-feature-gaps

Phase 3 — Feature Gap Implementation — Implementation Plan

> **For agentic workers:** REQUIRED SUB-SKILL: Use superpowers:subagent-driven-development or superpowers:executing-plans. Steps use checkbox (`- [ ]`) syntax for tracking.

Goal: Implement all missing features from `GAP_ANALYSIS.md`: LiteLLM abstraction, RepoMap, quality scoring, clarification system, plugin system, and 4 new providers (Cohere, Replicate, Together.ai, verify HuggingFace).

Architecture: Each feature is a self-contained new package under `internal/` with its own types, implementation, tests, and challenge. Packages do not depend on each other. Each follows the existing pattern: `types.go` + `implementation.go` + `*_test.go` + challenge.

Tech Stack: Go 1.24, tree-sitter, fsnotify, json, net/http

Spec: `docs/superpowers/specs/2026-05-08-helixcode-zero-bluff-completion-design.md`

File Structure Map

```
# LiteLLM
helix_code/internal/llm/litellm/types.go              — create
helix_code/internal/llm/litellm/unified_provider.go   — create
helix_code/internal/llm/litellm/adapter_openai.go     — create
helix_code/internal/llm/litellm/adapter_anthropic.go  — create
helix_code/internal/llm/litellm/adapter_google.go     — create
helix_code/internal/llm/litellm/registry.go           — create
helix_code/internal/llm/litellm/cost_tracker.go       — create
helix_code/internal/llm/litellm/unified_provider_test.go — create

# RepoMap
helix_code/internal/repomap/types.go          — create
helix_code/internal/repomap/scanner.go        — create
helix_code/internal/repomap/map_builder.go    — create
helix_code/internal/repomap/watcher.go        — create
helix_code/internal/repomap/repomap_tool.go   — create
helix_code/internal/repomap/repomap_test.go   — create

# Quality Scoring
helix_code/internal/quality/types.go          — create
helix_code/internal/quality/scorer.go         — create
helix_code/internal/quality/gate.go           — create
helix_code/internal/quality/history.go        — create
helix_code/internal/quality/scorer_test.go    — create
```

```
# Clarification System
helix_code/internal/clarification/types.go     — create
helix_code/internal/clarification/engine.go    — create
helix_code/internal/clarification/question.go  — create
helix_code/internal/clarification/engine_test.go — create

# Plugin System
helix_code/internal/plugins/types.go           — create
helix_code/internal/plugins/manifest.go        — create
helix_code/internal/plugins/loader.go          — create
helix_code/internal/plugins/registry.go        — create
helix_code/internal/plugins/exec.go            — create
helix_code/internal/plugins/loader_test.go     — create

# New Providers
helix_code/internal/llm/providers/cohere/client.go     — create
helix_code/internal/llm/providers/replicate/client.go  — create
helix_code/internal/llm/providers/together/client.go   — create
```

Task P3-T01: LiteLLM Abstraction Layer

Files: Create `helix_code/internal/llm/litellm/` package (8 files)

- [ ] **Step 1: Create types** (`types.go`)

```go
package litellm

import (
    "context"
    "time"
    "dev.helix.code/internal/llm"
)

type ResponseFormat string
const (
    FormatOpenAI    ResponseFormat = "openai"
    FormatAnthropic ResponseFormat = "anthropic"
    FormatGoogle    ResponseFormat = "google"
)

type FormatAdapter interface {
    Format() ResponseFormat
    ConvertRequest(req *llm.LLMRequest) (interface{}, error)
    ConvertResponse(raw interface{}) (*llm.LLMResponse, error)
    ConvertStreamChunk(raw interface{}) (*llm.LLMStreamChunk,
        error)
}
```

```
type UnifiedProviderConfig struct {
    Adapter      FormatAdapter
    Endpoint     string
    APIKey       string
    Timeout      time.Duration
    DefaultModel string
    MaxTokens    int
    Temperature  float64
    CostPer1KIn  float64
    CostPer1KOut float64
}

type ProviderInfo struct {
    Name           string
    Format         ResponseFormat
    Endpoint       string
    AuthType       string
    DefaultModel   string
    SupportsStream bool
    MaxContext     int
    Models         []string
}

type CostTracker struct {
    TotalCost     float64
    TotalTokens   int64
    TotalRequests int64
    BudgetLimit   float64
}
```

- [ ] **Step 2: Create OpenAI adapter** (`adapter_openai.go`)

Implements `FormatAdapter` for OpenAI-compatible APIs. Maps `llm.LLMRequest` → OpenAI JSON format, parses OpenAI response → `llm.LLMResponse`, handles stream chunks with `choices[0].delta.content`.

- [ ] **Step 3: Create Anthropic adapter** (`adapter_anthropic.go`)

Maps to Anthropic Messages API format. Handles `content_block_delta` and `message_stop` stream events.

- [ ] **Step 4: Create Google adapter** (`adapter_google.go`)

Maps to Gemini `generateContent` format. Handles `candidates[0].content.parts[0].text`.

- [ ]

Step 5: Create UnifiedProvider (unified_provider.go)

```
type UnifiedProvider struct {
    config UnifiedProviderConfig
    client *http.Client
    cost   *CostTracker
}

func (p *UnifiedProvider) Generate(ctx context.Context, req
        *llm.LLMRequest) (*llm.LLMResponse, error) {
    // 1. Convert to provider-native format via adapter
    // 2. Marshal to JSON
    // 3. HTTP POST with Bearer auth
    // 4. Parse response via adapter.ConvertResponse
    // 5. Track cost
}

func (p *UnifiedProvider) GenerateStream(ctx context.Context,
        req *llm.LLMRequest) (<-chan *llm.LLMStreamChunk, error)
        {
    // Similar but returns SSE stream channel
}
```

Full implementation in design spec §5.1 (P3-T01 Step 5).

- [ ] **Step 6: Create ProviderRegistry** (registry.go)

```
type Registry struct {
    mu       sync.RWMutex
    providers map[string]ProviderInfo
    adapters  map[string]FormatAdapter
}

func (r *Registry) Register(info ProviderInfo, adapter
        FormatAdapter) { ... }
func (r *Registry) GetProvider(provider string)
        (*UnifiedProvider, error) { ... }
func (r *Registry) ListProviders() []ProviderInfo { ... }
func (r *Registry) FromLLMsVerifier(ctx context.Context,
        verifierURL string) error { ... }
```

- [ ] **Step 7: Create CostTracker** (cost_tracker.go)

```
func (c *CostTracker) TrackUsage(in, out int) { /* compute cost
    */ }
func (c *CostTracker) OverBudget() bool { return c.TotalCost >
    c.BudgetLimit }
func (c *CostTracker) Reset() { ... }
```

- [ ] **Step 8: Write tests** (`unified_provider_test.go`)

```
func TestUnifiedProvider_OpenAIAdapter_Roundtrip(t *testing.T) {
    adapter := &OpenAIAdapter{}
    req := &llm.LLMRequest{Messages: []llm.Message{{Role:
        "user", Content: "hello"}}}
    converted, err := adapter.ConvertRequest(req)
    assert.NoError(t, err)
    convMap := converted.(map[string]interface{})
    assert.Equal(t, "user", convMap["messages"].([]openaiMsg)
        [0].Role)
}
```

- [ ] **Step 9: Build, test, commit**

```
cd HelixCode && go build ./internal/llm/litellm/... && go test ./
    internal/llm/litellm/ -count=1
cd /run/media/milosvasic/DATA4TB/Projects/HelixCode
git add helix_code/internal/llm/litellm/
git commit -m "feat(P3-T01): add LiteLLM unified provider
    abstraction layer

Phase: 3  Task: P3-T01"
```

Task P3-T02: RepoMap — Semantic Codebase Mapping

Files: Create `helix_code/internal/repomap/` package (6 files)

- [ ] **Step 1: Create types** (`types.go`)

```
package repomap

type SymbolNode struct {
    Name     string        `json:"name"`
    Kind     string        `json:"kind"` // function, type,
        import, variable
    File     string        `json:"file"`
    Line     int           `json:"line"`
    Children []*SymbolNode `json:"children,omitempty"`
    Doc      string        `json:"doc,omitempty"`
```

```
}

type RepoMap struct {
    Root    string         `json:"root"`
    Files   []string       `json:"files"`
    Symbols []*SymbolNode `json:"symbols"`
    Summary string
        `json:"summary"` // compressed view for LLM context
}

type RepoMapConfig struct {
    Root        string
    ExcludeDirs []string
    Languages   []string // go, python, typescript, rust, c
    MaxMapSize  int      // bytes
}
```

- [ ] **Step 2: Create scanner** (`scanner.go`)

Walks directories respecting `.gitignore`. Identifies files by language. Uses tree-sitter (`go-tree-sitter`) to parse each file into symbol tree. Extracts function signatures, type definitions, imports, and doc comments.

```
func (s *Scanner) Scan(ctx context.Context, cfg RepoMapConfig)
        (*RepoMap, error) {
    // Walk files -> filter by language -> tree-sitter parse ->
        extract symbols
}
```

- [ ] **Step 3: Create map builder** (`map_builder.go`)

Creates the compressed "repo map" summary string for LLM context. Prioritizes public symbols, extracts call graphs, limits size to `cfg.MaxMapSize`.

```
func (b *MapBuilder) Build(symbols []*SymbolNode, maxBytes int)
        string {
    // Build compressed hierarchical representation
    // Format: file_path:
    //           ClassName.funcName(arg: Type) → ReturnType
    //           imports: [pkg1, pkg2]
}
```

- [ ] **Step 4: Create watcher** (`watcher.go`)

Uses `fsnotify` to watch for file changes. Re-scans affected files only. Debounces changes (200ms).

- [ ]

Step 5: Create tool (`repomap_tool.go`)

Implements `tools.Tool` interface as `repomap` tool. Returns current repo map as JSON or compressed text.

- [ ] **Step 6: Write tests** (`repomap_test.go`)

```
func TestRepoMap_ScanGoProject(t *testing.T) {
    dir := t.TempDir()
    os.WriteFile(filepath.Join(dir, "main.go"), []byte("package
        main\nfunc main() {}"), 0644)
    rm, err := Scan(context.Background(), RepoMapConfig{Root:
        dir})
    require.NoError(t, err)
    assert.NotEmpty(t, rm.Files)
    assert.NotEmpty(t, rm.Symbols)
}
```

- [ ] **Step 7: Build, test, commit**

```
cd HelixCode && go build ./internal/repomap/... && go test ./
        internal/repomap/ -count=1
cd /run/media/milosvasic/DATA4TB/Projects/HelixCode
git add helix_code/internal/repomap/
git commit -m "feat(P3-T02): add RepoMap semantic codebase
        mapping with tree-sitter

Phase: 3  Task: P3-T02"
```

Task P3-T03: Confidence-Based Quality Scoring

Files: Create `helix_code/internal/quality/` package (5 files)

- [ ] **Step 1: Create types** (`types.go`)

```
package quality

type ScoreResult struct {
    Overall      float64           `json:"overall"` // 0-100
    Compilation  bool              `json:"compilation"`
    TestPassRate float64           `json:"test_pass_rate"`
    LintScore    float64           `json:"lint_score"`
    Security     int               `json:"security"`
    Details      map[string]string `json:"details"`
    Passed       bool              `json:"passed"`
}
```

```
type QualityGate struct {
    MinScore      float64 `yaml:"min_score"`
    RequireBuild  bool    `yaml:"require_build"`
    RequireTests  bool    `yaml:"require_tests"`
    RequireLint   bool    `yaml:"require_lint"`
}
```

- [ ] **Step 2: Create scorer** (`scorer.go`)

```
func (s *Scorer) Score(ctx context.Context, output string,
        codeDir string) (*ScoreResult, error) {
    // 1. Write output to temp file
    // 2. Try to compile (go build / npm build / cargo build)
    // 3. Run tests (go test / npm test / cargo test)
    // 4. Run linter (golangci-lint / eslint / clippy)
    // 5. Compute weighted score
}
```

- [ ] **Step 3: Create gate** (`gate.go`)

```
func (g *QualityGate) Check(result *ScoreResult) bool {
    if g.RequireBuild && !result.Compilation { return false }
    if g.RequireTests && result.TestPassRate < 1.0 { return
        false }
    if result.Overall < g.MinScore { return false }
    return true
}
```

- [ ] **Step 4: Create history** (`history.go`)

```
type History struct {
    Entries []ScoreResult `json:"entries"`
    Path    string        `json:"-"`
}
func (h *History) Append(r ScoreResult) error { ... }  // JSONL
        append
func (h *History) Average() ScoreResult { ... }        //
        aggregate
```

- [ ] **Step 5: Write tests, build, commit**

```
cd HelixCode && go test ./internal/quality/ -count=1
cd /run/media/milosvasic/DATA4TB/Projects/HelixCode
git add helix_code/internal/quality/
```

```
git commit -m
        "feat(P3-T03): add confidence-based quality scoring
        system

Phase: 3  Task: P3-T03"
```

Task P3-T04: Interactive Clarification System

Files: Create `helix_code/internal/clarification/` package (4 files)

☐ **Step 1: Create types** (`types.go`)

```
package clarification

type QuestionType string
const (
    YesNo       QuestionType = "yes_no"
    MultipleChoice QuestionType = "multiple_choice"
    FreeText    QuestionType = "free_text"
)

type Question struct {
    ID      string       `json:"id"`
    Type    QuestionType `json:"type"`
    Text    string       `json:"text"`
    Options []string     `json:"options,omitempty"`
    Default string       `json:"default,omitempty"`
}

type Answer struct {
    QuestionID string `json:"question_id"`
    Value      string `json:"value"`
}

type Session struct {
    ID        string     `json:"id"`
    Questions []Question `json:"questions"`
    Answers   []Answer   `json:"answers"`
    Context   string     `json:"context"`
    Timeout   time.Duration
}
```

☐ **Step 2: Create engine** (`engine.go`)

```
type Engine struct {
    sessions map[string]*Session
```

```
    mu       sync.RWMutex
}

func (e *Engine) DetectAmbiguity(prompt string) []Question {
    // Check for: missing file paths, vague requirements,
        conflicting constraints
    // Generate structured questions to resolve each ambiguity
}

func (e *Engine) Resolve(questions []Question, answers []Answer)
        string {
    // Build clarified prompt from questions + answers
}
```

- [ ] **Step 3: Create question generator** (`question.go`)

Pattern matching for common ambiguity types: - "fix the bug" → "Which file has the bug? What's the expected behavior?" - "make it faster" → "Which function? What's the current performance? What's the target?" - "add a feature" → "What inputs/outputs? Where in the codebase?"

- [ ] **Step 4: Write tests, build, commit**

```
cd HelixCode && go test ./internal/clarification/ -count=1
cd /run/media/milosvasic/DATA4TB/Projects/HelixCode
git add helix_code/internal/clarification/
git commit -m "feat(P3-T04): add interactive clarification engine

Phase: 3  Task: P3-T04"
```

Task P3-T05: Plugin/Extension System

Files: Create `helix_code/internal/plugins/` package (6 files)

- [ ] **Step 1: Create types** (`types.go`)

```
package plugins

type Plugin interface {
    Name() string
    Version() string
    Init(ctx context.Context) error
    Shutdown(ctx context.Context) error
    Tools() []tools.Tool
    Hooks() []hooks.Hook
    Commands() []commands.Command
```

```
}

type Manifest struct {
    Name         string            `yaml:"name"`
    Version      string            `yaml:"version"`
    Description  string            `yaml:"description"`
    Author       string            `yaml:"author"`
    APIVersion   string            `yaml:"api_version"`
    Dependencies []string          `yaml:"dependencies"`
    Capabilities []string          `yaml:"capabilities"`
    Entrypoint   string            `yaml:"entrypoint"`
    Sandbox      bool              `yaml:"sandbox"`
    Env          map[string]string `yaml:"env"`
}
```

- [ ] **Step 2: Create manifest parser** (`manifest.go`)

```
func ParseManifest(path string) (*Manifest, error) { /* YAML
        parse */ }
func (m *Manifest) Validate() error { /* check required fields,
        API version */ }
```

- [ ] **Step 3: Create loader** (`loader.go`)

```
type Loader struct {
    pluginDir string
    plugins   map[string]Plugin
    mu        sync.RWMutex
}

func (l *Loader) Load(ctx context.Context, manifestPath string)
        (Plugin, error) {
    // 1. Parse manifest
    // 2. Resolve dependencies (topological sort)
    // 3. If Sandbox: load via sandbox.Manager (reuse F14)
    // 4. Call Plugin.Init()
}
```

- [ ] **Step 4: Create registry** (`registry.go`)

```
type Registry struct {
    loader *Loader
    tools  tools.Registry
    hooks  hooks.Registry
}
```

```
func (r *Registry) Register(plugin Plugin) error { /* merge
        tools/hooks/commands */ }
func (r *Registry) List() []Plugin { ... }
func (r *Registry) Get(name string) (Plugin, bool) { ... }
```

☐ **Step 5: Create sandboxed executor** (`exec.go`)

Reuses F14 `sandbox.Manager` for plugin isolation. Each plugin runs in its own sandboxed process with capability restrictions.

☐ **Step 6: Write tests, build, commit**

```
cd HelixCode && go test ./internal/plugins/ -count=1
cd /run/media/milosvasic/DATA4TB/Projects/HelixCode
git add helix_code/internal/plugins/
git commit -m "feat(P3-T05): add plugin/extension system with
        sandboxed execution

Phase: 3  Task: P3-T05"
```

Task P3-T06: Cohere Provider

Files: Create `helix_code/internal/llm/providers/cohere/client.go`

```
package cohere

const CohereBaseURL = "https://api.cohere.com/v1/chat"

type Client struct {
    apiKey  string
    baseURL string
    client  *http.Client
}

func NewClient(apiKey string) *Client {
    return &Client{apiKey: apiKey, baseURL: CohereBaseURL,
        client: &http.Client{Timeout: 30 * time.Second}}
}

func (c *Client) Generate(ctx context.Context, req
        *llm.LLMRequest) (*llm.LLMResponse, error) {
    body := map[string]interface{}{
        "model":        req.Model,
        "message":      req.Messages[len(req.Messages)-1].Content,
        "chat_history":
        buildHistory(req.Messages[:len(req.Messages)-1]),
        "max_tokens":   req.MaxTokens,
```

```
        "temperature": req.Temperature,
    }
    // HTTP POST to c.baseURL with Bearer c.apiKey
    // Parse response
}
```

- [ ] **Test, build, commit:**

```
cd HelixCode && go build ./internal/llm/providers/cohere/
cd /run/media/milosvasic/DATA4TB/Projects/HelixCode
git add helix_code/internal/llm/providers/cohere/
git commit -m "feat(P3-T06): add Cohere LLM provider

Phase: 3  Task: P3-T06"
```

Task P3-T07: Replicate Provider

Files: Create `helix_code/internal/llm/providers/replicate/client.go`

Replicate uses a different API pattern (async predictions). Client calls `/v1/predictions` with model + input, polls for completion.

- [ ] **Test, build, commit:**

```
cd HelixCode && go build ./internal/llm/providers/replicate/
cd /run/media/milosvasic/DATA4TB/Projects/HelixCode
git add helix_code/internal/llm/providers/replicate/
git commit -m "feat(P3-T07): add Replicate LLM provider

Phase: 3  Task: P3-T07"
```

Task P3-T08: Together.ai Provider

Files: Create `helix_code/internal/llm/providers/together/client.go`

OpenAI-compatible endpoint at `https://api.together.xyz/v1/chat/completions`. Minimal adapter on top of existing OpenAI format.

- [ ] **Test, build, commit:**

```
cd HelixCode && go build ./internal/llm/providers/together/
cd /run/media/milosvasic/DATA4TB/Projects/HelixCode
git add helix_code/internal/llm/providers/together/
git commit -m "feat(P3-T08): add Together.ai LLM provider

Phase: 3  Task: P3-T08"
```

Task P3-T09: Verify HuggingFace Provider

Files: Check `helix_code/internal/llm/providers/huggingface/` (if exists)

```
grep -rn "simulated\|stub\|placeholder\|TODO" helix_code/
        internal/llm/providers/huggingface/ 2>/dev/null && echo
        "ISSUE FOUND" || echo "clean"
```

If stubbed: fix with real HuggingFace Inference API calls. If real: document as verified.

- [ ] **Commit:**

```
cd /run/media/milosvasic/DATA4TB/Projects/HelixCode
git add helix_code/internal/llm/providers/huggingface/
git commit -m
        "feat(P3-T09): verify HuggingFace provider is real, fix
        if stubbed

Phase: 3  Task: P3-T09"
```

Task P3-T10: Update GAP_ANALYSIS.md with completions

- [ ] **Step 1: Mark all Phase 3 implementations as COMPLETE in GAP_ANALYSIS.md**

```
cd /run/media/milosvasic/DATA4TB/Projects/HelixCode
# Update provider count, add evidence links
# Mark LiteLLM, RepoMap, plugins, quality, clarification as DONE
```

- [ ] **Step 2: Commit**

```
git add GAP_ANALYSIS.md HELIXCODE_FEATURE_GAP_ANALYSIS.md
git commit -m "docs(P3-T10): update gap analysis with Phase 3
        completions

Phase: 3  Task: P3-T10"
```

Task P3-T11: Phase 3 build verification

- [ ] **Step 1: Full build**

```
cd HelixCode && go build ./...
```

Expected: exit 0

- [ ] **Step 2: Full test suite**

```
cd HelixCode && go test -short ./...
```

Expected: PASS (or SKIP-OK)

- [ ] **Step 3: Anti-bluff sweep for new code**

```
cd HelixCode
grep -rn "simulated\|placeholder\|stub\|TODO" internal/llm/
        litellm/ internal/repomap/ internal/quality/ internal/
        clarification/ internal/plugins/ | grep -v test | grep
        -v doc || echo "clean"
```

Expected: `clean`

- [ ] **Step 4: Commit and push**

```
cd /run/media/milosvasic/DATA4TB/Projects/HelixCode
git add . && git commit -m
        "chore(P3-T11): Phase 3 complete — all feature gaps
        implemented

Phase: 3  Task: P3-T11  Evidence: go build ./... PASS"
git push github main
```

Phase 3 Completion Checklist

- [ ] LiteLLM unified provider with 3 format adapters
- [ ] RepoMap with tree-sitter scanning
- [ ] Quality scoring with gates
- [ ] Clarification engine
- [ ] Plugin system with sandboxed loading
- [ ] Cohere, Replicate, Together.ai providers
- [ ] HuggingFace verified real
- [ ] GAP_ANALYSIS.md updated

- [ ] `go build ./...` exits 0
- [ ] New code anti-bluff sweep clean
- [ ] Continue to Phase 4